

DESAFIO PRÁTICO

Modelando um Agente Preditor de Consumo

Regressão aplicada a dados reais — em duplas

Prof. Marcelo Amorim · Inteligência Artificial e Aprendizagem de Máquina · 2026/02

A missão

Vocês são uma dupla de cientistas de dados contratados por uma montadora.

A missão: construir um modelo capaz de prever o consumo de combustível (mpg) de um carro, só a partir das características técnicas dele — sem precisar rodar o carro de verdade.

Ao final, cada dupla vai reportar o desempenho do próprio modelo — o ranking sai do resultado real, não do achismo.

O dataset de vocês

O arquivo mpg_desafio_alunos.csv está disponível no Moodle, no tópico da Aula 4.

São 339 carros reais, fabricados entre 1970 e 1982 — dados verdadeiros, não inventados.

Baixem o arquivo agora e carreguem no Colab antes de continuar.



Os campos do dataset

mpg	Consumo (milhas/galão) — o que vamos PREVER (y)
cylinders	Número de cilindros do motor
displacement	Cilindrada do motor
horsepower	Potência do motor (tem valores faltando!)
weight	Peso do carro
acceleration	Tempo de aceleração
model_year	Ano do modelo (70 a 82)
origin	País de fabricação (usa/europe/japan)
name	Nome do carro — não usar como feature numérica



As regras do desafio

Trabalho em duplas.

y é sempre mpg. X é a feature que vocês escolherem entre horsepower, weight ou cylinders.

Escolham o random_state que quiserem no train_test_split — esse número vai virar parte da descoberta no final.

Sim, isso significa que duplas podem ter R^2 diferentes só por causa do seed — vamos discutir isso no final.

O roteiro do desafio

- 1 Explorar os dados
- 2 Escolher a feature
- 3 Tratar valores nulos
- 4 Separar treino e teste
- 5 Treinar a regressão linear
- 6 Avaliar (MAE, MSE, R^2)
- 7 Tentar melhorar (polinomial)
- 8 Decidir e justificar
- 9 Avaliação final



CHECKPOINT 1 de 9

Explorar os dados

Carreguem o CSV e explorem: quantas linhas? quais colunas? tem nulo? qual a correlação de cada variável numérica com mpg?

LEMBRETE DE SINTAXE

```
df = pd.read_csv("mpg_desafio_alunos.csv")  
df.info()  
df.corr(numeric_only=True)["mpg"]
```



Escolher a feature

Com base na correlação (não no achismo), escolham 1 feature entre horsepower, weight ou cylinders pra ser o X.

LEMBRETE DE SINTAXE

```
X = df[["weight"]] # troquem pela feature escolhida  
y = df["mpg"]
```


Tratar valores nulos

Se a feature escolhida tiver valores faltando (horsepower tem 6), tratem antes de continuar.

LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["horsepower"])  
# ou, alternativamente:  
df["horsepower"] = df["horsepower"].fillna(df["horsepower"].mean())
```



Separar treino e teste

Separem treino e teste. Escolham QUALQUER número pro `random_state` — anotem qual número usaram.

LEMBRETE DE SINTAXE

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=SEU_NUMERO_AQUI  
)
```



Treinar a regressão linear

Treinem uma regressão linear simples com os dados de treino.

LEMBRETE DE SINTAXE

```
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
modelo.fit(X_train, y_train)
```



CHECKPOINT 6 de 9

Avaliar (MAE, MSE, R²)

Calcule as 3 métricas no CONJUNTO DE TESTE de vocês (não o final ainda).

LEMBRETE DE SINTAXE

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pred = modelo.predict(X_test)
r2_score(y_test, pred)
```



Tentar melhorar (polinomial)

Testem PolynomialFeatures com grau 2 e grau 3. O R^2 melhorou?

LEMBRETE DE SINTAXE

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```



Decidir e justificar

Escolham o modelo final (linear ou polinomial, e qual grau). Escrevam, numa célula de texto, por que escolheram esse — não vale só ‘foi o que deu maior R^2 ’.

Compararem o R^2 no treino vs. no teste: muito diferentes? Pode ser sinal de overfitting.

